

NBchat — 项目完整技术报告

一、项目概览

NBchat 是一款基于 Jakarta EE 10 的全功能在线聊天系统。它不是一个玩具 Demo——它拥有 82 个 Java 源文件、17 张数据库表、3900 行 CSS 动效系统、2300 行 JavaScript 交互逻辑，总代码量超过 15,000 行。它支持从注册登录到私聊群聊、从朋友圈到语音通话、从消息反应到阅后即焚、从 AI 助手到 365 天聊天热力图的完整社交体验。

更重要的是，它已经运行在腾讯云轻量应用服务器 **175.178.56.39** 上。

二、模块架构拆解

整个系统可以像剥洋葱一样，从最外层到最内层分为 12 个模块。

模块一：配置与基础设施（config）

这是系统的“心脏起搏器”——它在应用启动的那一刻就开始工作。

Database.java 采用经典的 **Initialization-on-Demand Holder** 单例模式。类加载时，JVM 保证线程安全地创建 HikariCP 连接池。令人赞叹的是，它没有使用任何 XML 配置——数据库 URL、用户名、密码全部通过 **AppConfig.java** 优雅地注入。

AppConfig.java 的设计尤其巧妙：它采用**双层配置策略**。先读取 classpath 下的 **app.properties** 作为默认值，再用**环境变量覆盖**——把 key 中的 **.** 换成 **_** 并转大写。这意味着本地开发时你可以把密码写在文件里，但生产环境一行 **export DB_PASSWORD=xxx** 就能覆盖，密码永远不会进入版本控制。

SchemaMigrator.java 是一个迷你版的 Flyway。它在连接池初始化时自动运行，用 JDBC 元数据 API 检查列和表是否存在，按需执行 DDL。这意味着你永远不需要手动执行 **ALTER TABLE**——部署新版本后，数据库会自动适配。

DatabaseLifecycleListener.java 则是一个温柔的“送别者”——它监听 Servlet 容器的销毁事件，在 Tomcat 关闭时优雅地关闭连接池。

模块二：公共工具层（common）

这是整个系统的“工具箱”。每个工具都像一把瑞士军刀，小巧但不可或缺。

Jdbc.java —— 只有 56 行，却承载了整个系统的数据库交互。它用 Java 8 的函数式接口 **SqlConsumer<T>** 和 **RowMapper<T>** 封装了 JDBC 的样板代码。**insert()** 方法自动处理自增主键的返回，**list()** 和 **one()** 把 **ResultSet** 的遍历变成了优雅的 lambda 表达式。这让所有 DAO 层的代码看起来像在写声明式查询，而不是传统的 try-catch-finally 地狱。

Transactional.java —— 这是重构的胜利。最初三个 Service 各自复制了相同的 **withConnection** 模板代码。现在它被提取成一个独立的工具类，所有 Service 共享同一个连接管理入口。

RateLimiter.java —— 一个基于滑动窗口算法的频率限制器。它用 **ConcurrentHashMap** 保证线程安全，用 **ArrayDeque** 存储每个 key 的时间戳队列。登录接口每 5 分钟最多 8 次，验证码每 10 分钟最多 3 次——暴力破解在数学上变得不可能。

PasswordHasher.java —— 薄薄一层封装 BCrypt，但 12 轮的 salt rounds 意味着即使数据泄露，攻击者也无法反向推导密码。

SessionSupport.java —— 从 HTTP Session 中提取登录用户 ID 的简洁入口。它同时处理 **Long** 和 **Integer** 类型，因为不同的 Servlet 容器可能用不同的数值类型存储 session attribute。

模块三：数据模型层 (model)

17 个 Java **record** 类，每个都是一个不可变的数据快照。从 **User** 到 **ChatMessage**、从 **GroupMemberView** 到 **DailyMessageCount**，它们像乐高积木一样精确地映射数据库的每一行。

特别值得一提的是 **ChatMessage**——它从最初的 7 个字段逐步演进到 **14 个字段**：**replyToMessageId** 支撑引用回复，**replyPreview** 截取前 80 字预览，**reactions** 携带表情反应摘要，**recalledAt** 标记撤回时间。每一次字段的添加都伴随着一次 Schema 自动迁移。

VisibilityPolicy 是一个纯函数式的权限判定引擎。它用 Java 的 **switch** 表达式实现四种模式 (**ALL_FRIENDS** / **ONLY_SELF** / **SELECTED** / **EXCLUDE**)，配合 **Set** 交集运算，在 $O(n)$ 时间复杂度内完成朋友圈的可见性判断。

模块四：认证模块 (auth)

这是系统的“门禁系统”。注册、登录、密码重置三条主路径，每条路径都由 Resource → Service → DAO 三层串联。

注册流程像一场精心设计的仪式：用户先请求验证码 → **EmailCodeService** 生成 6 位随机数 → 存入 **email_verification_codes** 表（10 分钟有效期）→ **QqMailSender** 通过 QQ 邮箱 SMTP 发送 → 用户提交注册表单 → **AuthService** 校验验证码 → **AuthDao** 插入用户记录 → 自动创建默认好友分组 “My Friends” → 写入 HTTP Session 完成自动登录。

登录验证的设计尤为精妙：它**先查密码哈希，再比对**。这意味着即使用户名不存在，也会执行完整的 BCrypt 验证流程——防止通过响应时间差来枚举有效用户名（时序攻击防护）。

模块五：好友管理模块 (friend)

这个模块像一个社交网络的“关系织网机”。

好友请求是双向的——发送方看到“已发送”状态，接收方看到“收到请求”，各不干扰。**FriendDao.requests()** 通过一个动态 SQL 列名 (**sender_id** vs **receiver_id**) 实现同一查询的双重视角。

亲密好友 (close_friend) 机制是一个轻量级的特权标记。它被集成到群聊邀请策略中：如果被邀请人标记邀请人为亲密好友，则**直接入群**，无需经过邀请→接受的流程。**GroupInvitePolicy.decide()** 用一个简单的三元表达式实现了这个业务规则。

删除好友不是物理删除，而是将 **friendships.active** 置为 0，同时对双方共同的私聊消息执行 **message_visibility** 软删除——删除者看不到历史消息，但被删除者不受影响。这是一个非常细腻的隐私设计。

模块六：聊天核心模块（chat）

这是整个系统的"大脑"，也是最复杂的模块。`ChatDao.java` 有 475 行，是最大的单一文件。

会话列表查询是一个 SQL 大师级作品：一条 SQL 同时完成 PRIVATE/GROUP 分类、peer 信息关联、最后一条消息的子查询、未读计数（带 `conversation_reads` 和 `message_visibility` 双重过滤），最终按最后活动时间排序。整个查询涉及 5 个 JOIN 和 3 个子查询，但封装得非常干净。

历史消息查询采用游标分页而非传统的 OFFSET 分页。`ChatHistoryPageRequest` 携带一个 `beforeId`，查询 `WHERE m.id < ? ORDER BY m.id DESC LIMIT 50`。这意味着即使消息表达达到百万级，翻到最后一页的性能也是恒定的。

消息发送的权限检查经过了三次迭代。最初任何人都可以向任何会话发送消息。现在 `ensureCanSend()` 会验证：你是否在该会话中？如果是私聊，对方是否仍然是你的好友？每一步失败都有明确的中文错误提示。

SYSTEM 消息是隐形的记录者。群聊创建、成员加入/退出、角色变更、群名修改——这些事件都会在业务逻辑中自动触发 `saveSystemMessage()`，生成一条特殊的灰色消息插入聊天流中。消息内容使用 `displayName()` 而非裸 ID，人性化且可读。

消息撤回有一个严格的 2 分钟窗口。`recallMessage()` 的 SQL 条件 `sent_at >= DATE_SUB(NOW(), INTERVAL 2 MINUTE)` 在数据库层面保证了时间限制，无法被客户端绕过。撤回后消息类型变为 `RECALLED`，内容替换为"消息已撤回"，但不会从数据库中删除。

消息反应使用 `INSERT IGNORE` 实现幂等性——重复点击同一个 emoji 不会重复插入。`removeReaction` 直接 DELETE。反应的查询使用 `GROUP BY emoji` 聚合，同时用 `MAX(CASE WHEN user_id=? THEN 1 ELSE 0 END)` 判断当前用户是否已反应，一次查询返回所有需要的渲染数据。

模块七：朋友圈模块（moment）

朋友圈的实现解决了一个经典难题：**如何让不同的人看到不同的内容？**

`MomentDao.feed()` 的 SQL 长达 15 行，包含了 EXISTS 子查询和 NOT EXISTS 子查询，精确实现了四种可见性模式。`moment_visibility_rules` 表存储了每条朋友圈的 `SELECTED_FRIEND`、`SELECTED_GROUP`、`EXCLUDED_FRIEND`、`EXCLUDED_GROUP` 规则。`VisibilityPolicy.java` 作为纯内存计算引擎，在 Java 层也提供了相同的判定逻辑。

点赞使用了 `INSERT IGNORE`，天然防止重复。评论存储在 `moment_comments` 表中。`MomentCommentView` 是后来加入的——它让前端能从 `GET /moments/{id}/comments` 获取完整的评论列表（含用户昵称和时间）。

模块八：媒体上传模块（media）

图片、语音消息、头像、背景图——所有这些文件都通过统一的 `MediaResource` 入口上传。

`UploadKind` 枚举定义了五种上传类型，每种有不同的文件大小限制和 MIME 类型白名单。`MediaContentTypePolicy` 只允许 `image/jpeg`、`image/png`、`image/gif`、`image/webp` 等安全类型——即使攻击者试图上传一个伪装成图片的 `.exe`，也会在第一道防线被拦截。

`MediaService.save()` 的流程像一个安检传送带：先检查 Content-Type 是否合法 → 检查文件大小 → 用 UUID 重命名文件（防止路径遍历攻击）→ 写入磁盘 → 再次检查实际文件大小 → 写入数据库元数据。如果实际大小超出限制，已写入的文件会被删除并回滚。

`UploadStaticServlet` 是一个安全加固的静态文件服务器。它检查 `..` 防止目录遍历，用 `Path.normalize()` 和 `startsWith()` 做二次验证。

模块九：语音通话模块（voice）

语音通话是系统中最复杂的实时通信模块，它涉及两个通道：WebSocket（信令）和 WebRTC（媒体流）。

信令流程就像一场精心编排的三次握手：发起方调用 `POST /voice/calls/{calleeId}` → 服务器创建 `voice_call_sessions` 记录（状态 RINGING）→ 通过 Voice WebSocket 向对方发送 `call-invite` → 对方点击接听 → `POST /calls/{id}/accept` → 双方开始 WebRTC 协商（offer → answer → ICE candidate 交换）→ 媒体流通道建立。

`BusyUserRegistry` 使用 `ConcurrentHashMap.newKeySet()` 维护一个忙线用户集合。它的 `reserve()` 方法用原子操作保证两个用户同时只能在一通电话中——如果一方正在通话，另一方的呼叫会被直接拒绝。

`IceServerConfig` 支持通过 `app.properties` 配置 STUN/TURN 服务器。生产环境中配置 coturn 后，即使用户在 NAT 后面也能成功建立 P2P 连接。

模块十：WebSocket 实时推送（websocket）

两个 WebSocket Endpoint 分别服务聊天消息和语音信令，共享同一个 `SocketRegistry`。

`ChatEndpoint.onMessage()` 是一个事件路由器——它解析 JSON 中的 `event` 字段，分发到三条处理器：REACTION（表情反应）、RECALL（消息撤回）、普通消息。每种事件的处理结果都会通过 `SocketRegistry.send()` 广播给会话中的所有成员。

错误处理是这个端点最成熟的特性。如果消息发送失败（例如不是好友了还试图私聊），服务器不会默默吞掉异常，而是返回一个 `{"event": "ERROR", "message": "..."}` 的 JSON 对象。前端收到后会在消息列表中渲染一条红色感叹号的失败标记。

`HttpSessionConfigurator` 是 WebSocket 与 HTTP Session 之间的桥梁。WebSocket 协议本身没有 Session 概念，但通过这个配置器，WebSocket 连接可以访问到同一个用户的 HTTP Session，从而获取登录用户 ID——实现了 REST 和 WebSocket 的身份统一。

模块十一：AI 助手模块（ai）

Ollama 本地模型 → DeepSeek 云端 API。这个模块经历了一次关键技术选型变更。

`DeepSeekClient` 遵循 OpenAI 兼容的 `/chat/completions` 接口。它用 `java.net.http.HttpClient`（Java 11 内置，零外部依赖）发送 POST 请求，携带 `Authorization: Bearer {apiKey}` 头和 `system/user` 双消息结构。45 秒的超时设置比 Ollama 的 30 秒更宽容——云 API 的延迟天然更高。

`AiAssistantService` 是一个提示词工程师。它提取 `@千问小助手` 后面的文本作为用户问题，取最近 30 条群聊记录作为上下文，构造一个精心设计的中文 system prompt：“你是群聊中的中文智能助手...请基于群聊上文回答用户问题，回答要准确、简洁、可执行。”

模块十二：前端动效与交互系统

Canvas 粒子系统——一个定制的粒子引擎，由 `requestAnimationFrame` 驱动。发送消息时，34 个彩色粒子从发送按钮位置以扇形状炸出，每个粒子受重力和摩擦力影响，在 46 帧后自然消散。自动响应 `prefers-reduced-motion: reduce`。

阅后即焚刮刮乐——Canvas 的 `globalCompositeOperation: "destination-out"` 实现了真正的像素级擦除。18% 的透明度阈值触发自动销毁倒计时（8.5 秒）。支持鼠标和触摸事件，移动端可用。

聊天热力图——365 个格子组成 7×52 的网格，每日消息数用 `Math.ceil(Math.log2(count + 1))` 映射为 0~4 五级色阶。数据来自 `dailyMessageCounts()` 的 SQL 聚合查询。

昼夜主题切换——`localStorage` 持久化偏好 + CSS 类切换 + 0.48 秒全局过渡动画。导航栏的山水插画也随主题变化——月亮替换太阳，星星浮现。

@提及自动补全——`mentionAutocomplete` 状态机管理查询词、光标位置、候选列表。输入 @ 后实时过滤群成员昵称，支持中文匹配，最多显示 6 条建议。

三、数据库设计哲学

17 张表可以分为四个圈层：

圈层	表	职责
用户圈	users, email_verification_codes, password_reset_codes	身份与安全
社交圈	friend_groups, friend_requests, friendships	关系网络
内容圈	conversations, messages, message_visibility, message_reactions, conversation_reads, chat_groups, chat_group_members, chat_group_invitations	聊天核心
媒体圈	media_files, moments, moment_media, moment_visibility_rules, moment_likes, moment_comments, voice_call_sessions	富媒体扩展

每个圈层都可以独立扩展而不影响其他圈层。`conversations` 表通过 `type` 字段（PRIVATE/GROUP）实现多态——同一张表承载两种完全不同的聊天类型。

四、完整开发周期

阶段 0：骨架搭建（Day 0）

从 `mvn archetype:generate` 生成的空白 WAR 项目开始。删除了模板代码，建立 `com.example.chat` 包结构，手写 `pom.xml` 引入 Jakarta EE 10 全家桶——Servlet 6.0、WebSocket 2.1、Jersey REST 3.1.8、HikariCP 5.1.0、MySQL Connector 8.4.0、jBCrypt、Angus Mail。

阶段 1：认证系统（Day 1~2）

先写 `Jdbc.java`（整个系统的地基），再写 `AppConfig.java` + `Database.java`（配置层），然后自底向上写 `UserDao` → `UserService` → `UserResource`。认证是登录/注册/找回密码三条线，每条线都遵循 `Resource` → `Service` → `DAO` 的调用链。

阶段 2：好友系统（Day 3）

好友搜索、好友请求、好友分组、好友删除。这个阶段定义了 `friendships` 表的关键设计——`owner_id` 和 `friend_id` 的双向关系，以及 `active` 软删除标记。

阶段 3：聊天核心（Day 4~6）

这是最密集的阶段。先建 `conversations` 和 `messages` 表，再写 ChatDao 的私聊创建逻辑。然后接入 WebSocket——这是系统从“能用”到“实时”的质变点。`ChatEndpoint` 和 `SocketRegistry` 搭建了消息推送管道。

阶段 4：群聊（Day 7~8）

群聊是在私聊基础上的多对多扩展。`chat_groups` 和 `chat_group_members` 两张新表承载了群组元数据和成员关系。群聊邀请系统从最原始的“直接加入”演进到“亲密好友直加、普通好友发邀请”的策略模式。

阶段 5：朋友圈 + 媒体上传（Day 9~10）

朋友圈的可见性规则是这个阶段的核心挑战。四种模式的 SQL 查询需要 EXISTS 和 NOT EXISTS 的精妙配合。媒体上传系统同时服务聊天图片、朋友圈图片、头像、背景图五种场景。

阶段 6：语音通话 + AI 助手（Day 11~12）

WebRTC 信令是最复杂的实时通信模式。offer/answer/ICE-candidate 三段式协商 + 来电弹窗 UI + BusyUserRegistry 忙线管理。AI 助手最初用本地 Ollama，后来因为 2G 服务器装不下 7B 模型，改为云端 DeepSeek API。

阶段 7：消息反应 + 撤回 + 引用回复（Day 13~14）

这三个功能代表聊天体验从“能用”到“好用”的跨越。每个都涉及数据库 schema 变更（`message_reactions` 表、`recalled_at` 列、`reply_to_message_id` 列），由 `SchemaMigrator` 自动执行。

阶段 8：动效系统 + 热力图 + 阅后即焚 + 语音消息（Day 15~18）

前端体验的集中爆发期。Canvas 粒子引擎、qubic-bezier 弹簧动画、@脉冲高亮、图片渐进加载、爱心粒子、刮刮乐、365 天热力图——这些全部用原生 CSS/JS 实现，零第三方动画库。

阶段 9：部署上线（Day 19~20）

代码推送到 GitHub，腾讯云轻量服务器就绪。SSH 登录 → 安装 JDK 17 + MySQL 8 + Tomcat 10.1 → 导入 schema → 上传 WAR → 启动。`deploy.bat` 一键部署脚本把本地打包→上传→重启压缩为单条命令。

五、技术亮点速览

亮点	说明
Schema 自动迁移	应用启动时自动检测并创建缺失的列和表
环境变量覆盖配置	classpath 配置 + env var 双层注入
游标分页	百万级消息量下的恒定翻页性能
滑动窗口限流	ConcurrentHashMap + Deque 的精准频率控制

亮点	说明
三级角色权限	OWNER/ADMIN/MEMBER 精确到单个操作的鉴权
消息撤回 2 分钟窗口	数据库层面的时间约束，不可绕过
朋友圈四模式可见性	EXISTS/NOT EXISTS 子查询精准实现
Canvas 粒子系统	requestAnimationFrame + 物理模拟 + 无障碍适配
阅后即焚刮刮乐	Canvas destination-out + 透明度阈值 + 自动销毁
HiDPI 适配	Canvas 和热力图均使用 devicePixelRatio 渲染
零前端框架	2300 行原生 JS，无 React/Vue/Angular
零 XML 配置	纯 Java 注解 + 代码配置

六、数据一览

生产代码：	13,000+ 行
测试代码：	1,500+ 行
Java 文件：	82 个
JS 文件：	8 个
CSS 文件：	2 个
数据库表：	17 张
REST 端点：	28 个
WebSocket：	2 个
浏览器兼容：	Chrome/Firefox/Edge/Safari
部署平台：	腾讯云轻量服务器 Ubuntu 22.04